

# Unit Testing

---

## Preface:

Every Salesforce developer should know what a unit test is since they are required for production deployments, however, unit tests are not specific to Salesforce or Apex, they are an industry standard for just about every programming language and should be treated as such. Salesforce requires that 75% of code is covered in unit tests and that those unit tests pass in order to move code to production. This is good in theory since it forces developers to actually write the tests, however it has a pretty big side-effect in that many developers treat unit tests as an afterthought often writing them at the last minute, and writing bad tests just to get the required coverage.

The main purpose of unit tests is to make sure your code works and it's just as important as Production code. If there is a bug in production code, the very first step in debugging should be to run the associated unit tests to ensure they are still passing. A poorly written unit test is as bad if not worse than no test at all since it could result in false positive or false negative cases leading a developer to spend unnecessary time digging through the code to solve the issue.

Unit tests are specific to code so they can not and should not account for every single scenario that can happen with the application or system. Manual or functional tests are still required to account for other scenarios outside of the domain of the code itself.

There are a few different strategies for writing unit tests. One popular strategy being Test Driven Development (TDD) where the developer writes the test before they write the production code. This can further be broken down following 3 rules for TDD:

1. Write no production code except to pass a failing test
2. Write only enough of a test to demonstrate a failure
3. Write only enough production code to pass the test

## F.I.R.S.T

---

Clean tests follow five rules that form the above acronym:

**Fast:** Tests should be fast. They should run quickly. When tests run slow, you won't want to run them frequently. If you don't run them frequently, you won't find problems early enough to fix them easily. You won't feel as free to clean up the code. Eventually the code will begin to rot.

**Independent:** Tests should not depend on each other. One test should not set up the conditions for the next test. You should be able to run each test independently and run the tests in any order you like. When tests depend on each other, then the first one to fail causes a cascade of downstream failures, making diagnosis difficult and hiding downstream defects.

**Repeatable:** Tests should be repeatable in any environment. You should be able to run the tests in the production environment, in the QA environment, and on your laptop while riding home on the train without a network. If your tests aren't repeatable in any environment, then you'll always have an excuse for why they fail. You'll also find yourself unable to run the tests when the environment isn't available.

**Self-Validating:** The tests should have a boolean output. Either they pass or fail. You should not have to read through a log file to tell whether the tests pass. You should not have to manually compare two different text files to see whether the tests pass. If the tests aren't self-validating, then failure can become subjective and running the tests can require a long manual evaluation.

**Timely:** The tests need to be written in a timely fashion. Unit tests should be written just before the production code that makes them pass. If you write tests after the production code, then you may find the production code to be hard to test. You may decide that some production code is too hard to test. You may not design the production code to be testable.

### What a test should do:

Remember that the primary reason for unit tests is to make sure the code works and provide an outlet for easy debugging, NOT to reach a certain coverage % so that you can deploy. Unit tests should follow the S.O.L.I.D. principles just like production code. And following the Single Responsibility Principle, a unit test should test one and only one unit of code. Writing clean production code will usually result in a clean test. If following TDD, writing clean tests will even more frequently result in clean production code. Take the following code example:

```
public void changeAccountOwnership(String accountId) {
    Account a = [SELECT Id, OwnerId FROM Account WHERE Id =: accountId];
    a.OwnerId = UserInfo.getUserId();
    update a;
}
```

Many tests for this would look something like:

```

@Test static void testAccount() {
    User u = new User();
    u.FirstName = 'Test';
    //setting required fields
    insert u;

    User u2 = new User();
    u2.FirstName = 'Test2'
    //setting required fields

    Account a = new Account();
    a.Name = 'Test';
    a.OwnerId = u2.Id;
    //setting required fields
    insert a;

    System.runAs(u) {
        Test.startTest();
        AccountController.changeAccountOwnership(a.Id);
        Test.stopTest();

        Account queriedAccount = [SELECT Id, OwnerId FROM Account WHERE Id =:a.Id];
        System.assertEquals(u.Id, queriedAccount.OwnerId);
    }
}

```

There are a few issues with this code; the method `changeAccountOwnership()` is breaking the Single Responsibility Principle as it is not just responsible for changing the Account owner. It is also responsible for querying the Account and updating it. Because of this the unit test also has to worry about setting up data and interacting with the database. What this can result in is a false negative test run. Let's assume this code makes its way to production and then a validation rule is added on the User record. This unit test will now fail if not updated properly, but the failure has nothing to do with the unit under test. Also, every other unit test that creates test Users will also fail. Having possibly hundreds of unit tests failing just because of a simple validation rule can cause a lot of overhead and unnecessary time fixing and deploying unit tests. The unit test also spends a lot of resources just setting up data that could cause these issues. Unless the responsibility of the unit under test specifically interacts with the database, the unit test should not do so. This can cause the test to take longer to run since it needs to create then destroy data, and this problem is exponential as more tests are created this way. Instead, mock objects should be created. This will ensure that the unit test won't fail unless there is a direct issue with the production code. This will also cause the unit test to run faster since no creation and cleanup is necessary from the backend. The unit test name `testAccount()` is also not descriptive so when a user runs the test and views the output log, it would be difficult to tell what the test is supposed to do.

Let's take a look at how to write better production code and a better test:

Production Code:

```

public AccountDAO accountAccessor = new AccountDataAccessor();
public UserDAO userAccessor = new userDataAccessor();
public String userId = userAccessor.getCurrentUserId();

public void changeAccountOwnership(String accountId) {
    Account account = this.accountAccessor.getAccountFromId(accountId);
    account.OwnerId = userId;
    this.accountAccessor.updateRecords(new List<Account>{account});
}

```

Unit Test:

```

private static User buildMockUser(String userId) {
    //MockSObjectBuilder is a class that can generate an SObject using JSON without inserting anything into the database while still
    MockSObjectBuilder mockBuilder = new MockSObjectBuilder(User.sObjectType);
    mockBuilder.setField('Id', userId);
    //setting other fields
    User testUser = (User) mockBuilder.build();

    return testUser;
}

private static Account buildMockAccount(String accountId, String userId) {
    MockSObjectBuilder mockBuilder = new MockSObjectBuilder(Account.sObjectType);
    mockBuilder.setField('Id', accountId);
    mockBuilder.setField('OwnerId', userId);
}

```

```

//setting other fields
Account testAccount = (Account) mockBuilder.build();

return testUser;
}

@isTest static void changeAccountOwnership_givenNewUser_accountOwnerShouldUpdate() {
    String testUserId = MockIdGenerator.getMockId(User.sObjectType);
    String testUserId2 = MockIdGenerator.getMockId(User.sObjectType);
    String testAccountId = MockIdGenerator.getMockId(Account.sObjectType);

    User testUser = buildMockUser(testUserId);
    UserDataAccessorMock userMock = new UserDataAccessorMock();
    userMock.currentUser = testUser;

    Account testAccount = buildMockAccount(testAccountId, testUserId2);
    AccountDataAccessorMock accountMock = new AccountDataAccessorMock();
    accountMock.mockDatabaseAccounts = new List<Account>{testAccount};

    Test.startTest();
        AccountController.changeAccountOwnership(testAccountId);
    Test.stopTest();

    Account changedAccount = accountMock.getRecords()[0];

    System.assertEquals(testUserId, changedAccount.OwnerId);
}

@isTest static void changeAccountOwnership_givenInvalidAccount_exceptionShouldBeThrown() {
    String testUserId = MockIdGenerator.getMockId(User.sObjectType);
    String testUserId2 = MockIdGenerator.getMockId(User.sObjectType);
    String testAccountId = MockIdGenerator.getMockId(Account.sObjectType);
    String testAccountId2 = MockIdGenerator.getMockId(Account.sObjectType);

    User testUser = buildMockUser(testUserId);
    UserDataAccessorMock userMock = new UserDataAccessorMock();
    userMock.currentUser = testUser;

    Account testAccount = buildMockAccount(testAccountId, testUserId2);
    AccountDataAccessorMock accountMock = new AccountDataAccessorMock();
    accountMock.mockDatabaseAccounts = new List<Account>{testAccount};

    Boolean exceptionThrown = false;

    Test.startTest();
        try {
            AccountController.changeAccountOwnership(testAccountId2);
        } catch (Exception e) { //specific exception should be caught, not a generic Exception, this is just an example
            exceptionThrown = true;
        }
    Test.stopTest();

    System.assertEquals(true, exceptionThrown);
}

```

Much like @testSetup methods, the buildMockUser() and buildMockAccount() methods are used to create the mock records so that a database interaction is not needed. This test method now only checks to make sure the method it is testing changes the owner of the Account record. The test methods for the methods in Data Accessor classes should actually run DML and queries across the associated objects to account for backend interactions. This example also shows both a positive and negative test case. Unit tests should cover the full spectrum of possible outcomes of the code, not just what ideally should happen.

#### Recap:

- Clean tests are just as important as clean production code
- If your test is too long, your production code is probably too long
- Test methods should test one and only one unit of code
- Tests should handle negative as well as positive cases if applicable
- Mocking should be used to decouple tests from databases
- Database interaction should still happen with unit tests related to DAs so that full functionality is still covered

- TDD can be used to write better tests and production code